

SYSTEM ANALYST

presented by Ayub

SQL Basics

Зачем вообще базы данных?

- Инстаграм хранит **миллиарды** фото — где?
- Банк помнит **каждый** твой платёж — как?
- Spotify знает твою историю прослушиваний — в чём?

Всё это — базы данных. SQL — язык, на котором с ними разговаривают.

Что мы изучим сегодня

- Что такое таблица, строка, столбец
- Типы данных в SQL
- **CREATE TABLE** — создать таблицу
- **INSERT INTO** — добавить записи
- **SELECT** — получить данные
- **ALTER TABLE** — изменить структуру
- **UPDATE** — изменить записи
- **DELETE** — удалить записи

Вы уже рисовали Class Diagram — класс `User` с атрибутами `name`, `email`.

А как хранить тысячи таких пользователей в реальной системе?

Таблица = Класс. Строка = Объект.

- **Таблица (Table)** — структура хранения, как класс в UML
- **Столбец (Column)** — атрибут класса (`name` , `email`)
- **Строка (Row)** — конкретный объект / экземпляр (Алексей , `alex@mail.ru`)
- **Ячейка (Cell)** — одно конкретное значение (`"Алексей"`)

Пример: Class Diagram vs SQL Таблица

id	name	email	age
1	Алексей	alex@mail.ru	28
2	Мария	maria@gmail.com	24
3	Данияр	daniyar@yandex.kz	31

- Класс `User` описывает **что хранить**
- Таблица `users` **хранит** реальные данные
- Каждая строка — отдельный человек
- Первичный ключ (**Primary Key**) `id` — уникальный номер каждой строки

Module Check — Таблицы

1. **Чем таблица похожа** на Class Diagram?
2. **Что такое строка** в базе данных — своими словами?
3. **Зачем нужен** столбец `id` ?

**У нас есть поле `price` для цены товара.
Какой тип лучше: `INT` или `DECIMAL` ? Почему?**

Типы данных в SQL

Тип	Что хранит	Пример
INT	Целое число	ID: 42 , возраст: 28
VARCHAR(n)	Строка до n символов	Имя: 'Алексей'
TEXT	Длинный текст (без лимита)	Комментарий, описание
BOOLEAN	Истина / ложь	Активен: true
DATE	Дата	Дата рождения: 2000-03-15
DECIMAL(p,s)	Дробное число	Цена: 10.99
TIMESTAMP	Дата + время	Создан: 2024-01-15 14:30:00

VARCHAR vs TEXT — когда что использовать?

- **VARCHAR(n)** — строка с ограничением длины
 - Имя: **VARCHAR(100)** — имя не бывает длиннее 100 символов
 - Email: **VARCHAR(255)** — стандартный максимум
 - Используй когда знаешь максимальную длину
- **TEXT** — строка без ограничения
 - Комментарий пользователя
 - Описание товара
 - Биография профиля

*Правило: знаешь максимум → **VARCHAR**. Не знаешь → **TEXT**.*

Module Check — Типы данных

1. **Какой тип** выбрать для поля `phone_number` ?
2. **Чем** `DECIMAL` лучше `INT` для цены?
3. **Когда** использовать `TEXT` вместо `VARCHAR` ?
4. **Какой тип** для поля "дата регистрации пользователя"?

**Мы знаем какие поля нужны и какие у них типы.
Как теперь создать саму таблицу?**

CREATE TABLE — синтаксис

```
CREATE TABLE имя_таблицы (  
    имя_столбца ТИП_ДАНЫХ,  
    имя_столбца ТИП_ДАНЫХ,  
    ...  
);
```

- CREATE TABLE — команда создания
- Внутри скобок — список столбцов
- Каждый столбец: имя + тип через пробел
- Столбцы разделяются запятой
- В конце — ;

CREATE TABLE users

```
CREATE TABLE users (  
  id          INT,  
  name        VARCHAR(100),  
  email       VARCHAR(255),  
  age         INT,  
  is_active   BOOLEAN,  
  created_at  DATE  
);
```

- Таблица `users` — основа всего урока
- 6 столбцов с разными типами
- `is_active` — активен ли аккаунт
- `created_at` — дата регистрации

CREATE TABLE orders

```
CREATE TABLE orders (  
  id          INT,  
  user_id     INT,  
  product_name VARCHAR(200),  
  price       DECIMAL(10, 2),  
  order_date  DATE  
);
```

- `user_id` — ссылка на пользователя из `users`
- `DECIMAL(10, 2)` — число с двумя знаками после запятой
- Эти две таблицы будем использовать весь урок

Практика 1 — CREATE TABLE

Задача: создай таблицу `products`

Поле	Тип
<code>id</code>	INT
<code>name</code>	VARCHAR(200)
<code>price</code>	DECIMAL(10, 2)
<code>in_stock</code>	BOOLEAN
<code>added_date</code>	DATE

Практика 1 — шаги

1. Напиши `CREATE TABLE products (`
2. Добавь каждое поле с типом через запятую
3. Закрой скобку и поставь `;`

```
CREATE TABLE products (  
    id          INT,  
    name        VARCHAR(200),  
    price       DECIMAL(10, 2),  
    in_stock    BOOLEAN,  
    added_date  DATE  
);
```

**Таблица создана. Она пустая.
Как добавить туда первые данные?**

INSERT INTO — два варианта

Вариант 1 — без имён столбцов (опасно):

```
INSERT INTO users VALUES (1, 'Алексей', 'alex@mail.ru', 28, true, '2024-01-10');
```

Вариант 2 — с именами столбцов (безопасно):

```
INSERT INTO users (id, name, email, age, is_active, created_at)  
VALUES (1, 'Алексей', 'alex@mail.ru', 28, true, '2024-01-10');
```

Всегда используйте вариант 2 — если порядок столбцов изменится, данные не перепутаются.

Заполняем таблицу users

```
INSERT INTO users (id, name, email, age, is_active, created_at) VALUES
(1, 'Алексей', 'alex@mail.ru', 28, true, '2024-01-10'),
(2, 'Мария', 'maria@gmail.com', 24, true, '2024-02-15'),
(3, 'Данияр', 'daniyar@mail.ru', 31, false, '2023-11-01'),
(4, 'Айгерим', 'aigul@yandex.kz', 19, true, '2024-03-20'),
(5, 'Павел', 'pavel@inbox.ru', 65, false, '2022-07-08');
```

- 5 пользователей с разными данными
- `is_active = false` у некоторых — пригодится для фильтрации

Заполняем таблицу orders

```
INSERT INTO orders (id, user_id, product_name, price, order_date) VALUES
(1, 1, 'Наушники Sony', 3500.00, '2024-03-01'),
(2, 1, 'Ноутбук Lenovo', 85000.00, '2024-03-15'),
(3, 2, 'Мышь Logitech', 990.00, '2024-04-02'),
(4, 3, 'Клавиатура', 2200.00, '2024-04-10'),
(5, 4, 'Кабель USB', 150.00, '2024-04-12');
```

- `user_id` указывает кто сделал заказ
- Разные цены — пригодится для фильтрации по `WHERE`

Практика 2 — INSERT INTO

Задача: вставь 3 записи в таблицу `products`

Шаги:

1. Напиши `INSERT INTO products (name, price, in_stock, added_date)`
2. Добавь `VALUES` и первую строку данных
3. Повтори для 3 строк (можно в одном INSERT через запятую)

Примеры данных:

- Телефон Samsung, 45000.00, true, 2024-01-05
- Чехол для телефона, 800.00, true, 2024-01-10
- Зарядное устройство, 1200.00, false, 2023-12-20

**У нас 1000 пользователей в таблице.
Как найти только тех, кто старше 25 лет?**

SELECT — выбрать всё

```
SELECT * FROM users;
```

- `SELECT` — что выбрать
- `*` — все столбцы
- `FROM users` — из какой таблицы

Результат: все строки, все столбцы таблицы `users`

SELECT — конкретные столбцы

```
SELECT name, email FROM users;
```

Результат:

name	email
Алексей	alex@mail.ru
Мария	maria@gmail.com
Данияр	daniyar@mail.ru

- Получаем только нужные столбцы
- Быстрее и экономичнее чем `SELECT *`

SELECT WHERE — фильтрация

```
SELECT * FROM users WHERE is_active = true;
```

```
SELECT * FROM orders WHERE price > 500;
```

```
SELECT * FROM users WHERE age >= 25 AND is_active = true;
```

- `WHERE` — условие фильтрации
- `=`, `>`, `<`, `>=`, `<=` — операторы сравнения
- `AND` — оба условия должны быть истинны

Практика 3 — SELECT

Три мини-задачи:

1. Выбери все товары из `products` дороже 100
2. Выбери только `name` и `price` из `products`
3. Найди все товары в наличии (`in_stock = true`) дешевле 1500

Подсказка для задачи 3:

```
SELECT ... FROM products WHERE ... AND ...;
```

Module Check — SELECT

1. **Чем** `SELECT *` отличается от `SELECT name, email`?
2. **Что вернёт** `SELECT * FROM users WHERE age < 20`?
3. **Напиши запрос:** найди все заказы пользователя с `user_id = 1`
4. **Что значит** `AND` в WHERE-условии?

Таблица `users` уже в работе, тысячи записей.

Менеджер говорит: нужно добавить поле `phone`. Как?

ALTER TABLE — добавить столбец

```
ALTER TABLE users ADD COLUMN phone VARCHAR(20);
```

- ALTER TABLE — изменить структуру существующей таблицы
- ADD COLUMN — добавить новый столбец
- Данные в таблице **не пропадают**
- Новый столбец для старых строк будет NULL

Бизнес-требования меняются — ALTER TABLE позволяет адаптировать таблицу без потери данных.

Практика 4 — ALTER TABLE

Задача: добавь в таблицу `products` столбец `category`

Шаги:

1. Напиши `ALTER TABLE products`
2. Добавь `ADD COLUMN category VARCHAR(50);`
3. Проверь результат: `SELECT * FROM products`

Ожидаемый результат:

- Новый столбец `category` появился
- У всех существующих товаров `category = NULL`

Пользователь Данияр снова активен.

Как обновить его запись в таблице, не создавая новую?

UPDATE — обновить запись

```
UPDATE users SET is_active = true WHERE id = 3;
```

- `UPDATE` — какую таблицу менять
- `SET` — что изменить (столбец = новое значение)
- `WHERE` — какую строку менять

Обновить несколько столбцов сразу:

```
UPDATE orders  
SET price = 999.99, product_name = 'Ноутбук Pro'  
WHERE id = 2;
```

UPDATE без WHERE — это катастрофа

-- ОПАСНО: обновит ВСЕХ пользователей

```
UPDATE users SET is_active = false;
```

-- ПРАВИЛЬНО: только нужную строку

```
UPDATE users SET is_active = false WHERE id = 3;
```

Правило: перед каждым UPDATE всегда пиши WHERE. Проверяй два раза.

Практика 5 — UPDATE

Две мини-задачи:

1. Обнови цену товара с `id = 1` в `products` на `299.99`
2. Установи `is_active = false` для всех пользователей старше 60

Шаги для задачи 2:

1. Напиши `UPDATE users`
2. Добавь `SET is_active = false`
3. Добавь `WHERE age > 60`
4. Проверь: `SELECT * FROM users WHERE age > 60`

**Пользователь попросил удалить его аккаунт.
Как удалить конкретную строку из таблицы?**

DELETE — удалить строку

```
DELETE FROM users WHERE id = 5;
```

- `DELETE FROM` — из какой таблицы удалять
- `WHERE` — условие (какие строки удалить)
- Удалённые строки восстановить **нельзя**

DELETE vs DROP TABLE:

Команда	Что делает
<code>DELETE FROM users WHERE id = 5</code>	Удаляет одну строку
<code>DELETE FROM users</code>	Удаляет все строки, таблица остаётся
<code>DROP TABLE users</code>	Удаляет всю таблицу целиком

DELETE без WHERE — тоже катастрофа

```
-- ОПАСНО: удалит ВСЕ строки из таблицы
```

```
DELETE FROM orders;
```

```
-- ПРАВИЛЬНО: только нужную строку
```

```
DELETE FROM orders WHERE id = 5;
```

Правило: те же что для UPDATE — всегда проверяй WHERE перед DELETE.

Практика 6 — DELETE

Две задачи:

1. Удали товар с `id = 3` из `products`
2. Удали все заказы, где `price < 10`

Шаги для задачи 2:

1. Сначала проверь: `SELECT * FROM orders WHERE price < 10`
2. Убедись что удаляешь нужные строки
3. Напиши `DELETE FROM orders WHERE price < 10;`

Module Check — UPDATE и DELETE

1. **Что произойдёт** если написать `DELETE FROM users` без WHERE?
2. **Чем** `DELETE FROM users` отличается от `DROP TABLE users` ?
3. **Напиши запрос:** обнови email пользователя с id=2 на `new@mail.ru`
4. **Почему** нужно сначала писать SELECT перед DELETE?

Итог: все 6 команд

Команда	Что делает
CREATE TABLE	Создаёт новую таблицу
INSERT INTO	Добавляет строку в таблицу
SELECT	Читает данные из таблицы
ALTER TABLE	Изменяет структуру таблицы
UPDATE	Изменяет данные в строках
DELETE	Удаляет строки из таблицы

Большая практика — задача 1

Сценарий: Зарегистрировался новый пользователь Алинур, 28 лет.

Задача: добавь его в таблицу `users`

Шаги:

1. Определи следующий свободный `id` (у нас было 5 — берём 6)
2. Напиши `INSERT INTO users (id, name, email, age, is_active, created_at)`
3. Добавь `VALUES` с реальными данными
4. Придумай email и дату регистрации

Большая практика — задача 2

Сценарий: Алинур сменил email.

Задача: обнови его email в таблице

Шаги:

1. Напиши `SELECT * FROM users WHERE id = 6` — убедись что нашёл нужную строку
2. Напиши `UPDATE users SET email = 'новый@mail.ru'`
3. Добавь `WHERE id = 6`
4. Снова сделай `SELECT` и проверь изменение

Большая практика — задача 3

Сценарий: Менеджер хочет список активных пользователей моложе 30.

Задача: напиши запрос

Шаги:

1. Напиши `SELECT * FROM users`
2. Добавь `WHERE is_active = true`
3. Добавь `AND age < 30`

Кто должен попасть в результат?

- Алексей (28, active) — да
- Мария (24, active) — да
- Айгерим (19, active) — да

Большая практика — задача 4

Сценарий: Менеджер просит добавить статус заказа.

Шаги:

1. Добавь в таблицу `orders` столбец `status VARCHAR(20)`

```
ALTER TABLE orders ADD COLUMN status VARCHAR(20);
```

2. Установи статус `'new'` для всех заказов

```
UPDATE orders SET status = 'new';
```

(Здесь WHERE не нужен — меняем ВСЕ строки намеренно)

Большая практика — задача 5

Сценарий: Найди дорогие заказы и пометь их как VIP.

Шаги:

1. Сначала посмотри что есть:

```
SELECT * FROM orders WHERE price > 1000;
```

2. Обнови статус для этих заказов:

```
UPDATE orders SET status = 'vip' WHERE price > 1000;
```

3. Проверь результат: `SELECT * FROM orders;`

Большая практика — задача 6

Сценарий: Пользователь с id=3 попросил удалить все его заказы.

Шаги:

1. Посмотри его заказы:

```
SELECT * FROM orders WHERE user_id = 3;
```

2. Убедись что видишь только его заказы

3. Удали:

```
DELETE FROM orders WHERE user_id = 3;
```

4. Проверь: `SELECT * FROM orders WHERE user_id = 3;`

Final Check

1. **Как создать** таблицу `categories` с полями `id` (INT) и `name` (VARCHAR(100))?
2. **Какой тип данных** выбрать для поля "описание продукта" — длина заранее неизвестна?
3. **Что вернёт** `SELECT * FROM orders WHERE price BETWEEN 1000 AND 5000`?
4. **Напиши** UPDATE который установит `is_active = false` для пользователей с `age > 70`

Итоги урока

Таблица = класс из UML. **Строка** = объект. **Столбец** = атрибут.

Тип данных определяет что хранить: `INT`, `VARCHAR`, `DECIMAL`, `BOOLEAN`, `DATE`, `TIMESTAMP`.

CREATE TABLE создаёт структуру. **INSERT** заполняет. **SELECT** читает. **WHERE** фильтрует.

UPDATE и **DELETE** без `WHERE` — меняют и удаляют **ВСЕ** строки. Всегда проверяй условие.

ALTER TABLE — безопасно изменяет структуру без потери данных.